TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TẦNG 2 Chọn đội tuyển Tinh dự thi Quốc gia

SỐ HỌC
THUẬT TOÁN

Sàng tuyến tính · Hàm số học · Nghịch đảo modulo · Miller – Rabin
Tổ hợp mod p · Legendre · Lũy thừa ma trận · Digit DP

SÀNG 1..120 · gạch chéo theo ước nguyên tố nhỏ nhất

1	2	3	~~4~~	5	~~6~~	7	~~8~~	~~9~~	~~10~~	11	~~12~~
13	~~14~~	~~15~~	~~16~~	17	~~18~~	19	~~20~~	~~21~~	~~22~~	23	~~24~~
~~25~~	~~26~~	~~27~~	~~28~~	29	~~30~~	31	~~32~~	~~33~~	~~34~~	~~35~~	~~36~~
37	~~38~~	~~39~~	~~40~~	41	~~42~~	43	~~44~~	~~45~~	~~46~~	47	~~48~~
~~49~~	~~50~~	~~51~~	~~52~~	53	~~54~~	~~55~~	~~56~~	~~57~~	~~58~~	59	~~60~~
61	~~62~~	~~63~~	~~64~~	~~65~~	~~66~~	67	~~68~~	~~69~~	~~70~~	71	~~72~~
73	~~74~~	~~75~~	~~76~~	~~77~~	~~78~~	79	~~80~~	~~81~~	~~82~~	83	~~84~~
~~85~~	~~86~~	~~87~~	~~88~~	89	~~90~~	~~91~~	~~92~~	~~93~~	~~94~~	~~95~~	~~96~~
97	~~98~~	~~99~~	~~100~~	101	~~102~~	103	~~104~~	~~105~~	~~106~~	107	~~108~~
109	~~110~~	~~111~~	~~112~~	113	~~114~~	~~115~~	~~116~~	~~117~~	~~118~~	~~119~~	~~120~~

$a^{p-1} \equiv 1 \pmod p$

$\varphi(n) = n \prod (1 - 1/p)$
p chạy qua ước nguyên tố

$v_p(n!) = \Sigma \lfloor n / p^k \rfloor$
công thức Legendre

$C(n, k) = n! / (k!\,(n-k)!)$
trong O(1) theo mod p

$ax + by = gcd(a, b)$
Euclid mở rộng

$F(10^{18}) \bmod p \leftarrow [1\ 1;\ 1\ 0]^n$ 60 phép bình phương

01 Sàng nâng cao	02 Hàm số học	03 Modulo nâng cao	04 Tổ hợp mod p	05 Giai thừa và số mũ	06 Ma trận · Digit DP

TẬP 3 · HƯỚNG DẪN GIẢI — 60 lời giải · từ ngây thơ đến tối ưu · bảng điểm thật

Nguyên tố · Ước số · Đồng dư · Tổ hợp · Chữ số

Tổng hợp và biên soạn: Lê Thành Nam

Lê Thành Nam

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho **60 bài tập** của Tập 2. Mỗi lời giải không chỉ đưa ra chương trình đạt 100 điểm, mà còn chỉ ra vì sao công thức hay thuật toán ấy đúng, vì sao nó đủ nhanh, và vì sao những chương trình "trông hợp lí" khác lại sai hoặc chậm.

Mọi chương trình trong sách đều đã được kiểm tra hai lần: đối chiếu với lời giải vét cạn (nhiều khi viết bằng Python với số nguyên lớn) trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và chấm trên bộ test chính thức với giới hạn ghi trong đề. Các con số trong bảng điểm cuối mỗi bài là **điểm thực tế** mà từng chương trình nhận được, không phải ước lượng. Nhiều bảng điểm có cả những chương trình sai: quên trường hợp $m = 1$ hay $a \equiv 1$, quên ước nguyên tố cuối cùng lớn hơn $\sqrt{n}$, rút gọn số mũ theo p thay vì $p - 1$, dùng Fermat khi modulo không nguyên tố, quên dấu trừ, nhân tràn số, chia số âm bằng phép / của C++, ghi nhớ trạng thái tight trong Digit DP... Đó là những lỗi mà rất nhiều học sinh mắc phải, được giữ lại để em thấy chúng mất bao nhiêu điểm.

Một điều nổi bật khi chấm các bài số học: **nhiều chương trình sai vẫn đúng trên hầu hết dữ liệu ngẫu nhiên**. Một số ngẫu nhiên hiếm khi là số giả nguyên tố mạnh, hiếm khi đồng dư 1 theo $10^9 + 7$, hiếm khi có số mũ rút gọn bằng 0. Các lời giải luôn nói rõ test nào làm một chương trình mất điểm và vì sao.

Lê Thành Nam

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
Ý tưởng chính	Công thức hay thuật toán được dùng, kèm lập luận vì sao nó đúng và đủ nhanh (các mục đầu của mỗi lời giải).
Các cách chưa đúng	Từng chương trình sai hoặc chậm trong bảng điểm và lí do nó mất điểm.
Cách tối ưu	Chương trình hoàn chỉnh và độ phức tạp.
Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Lời khuyên: Đừng đọc lời giải như đọc truyện. Đọc mục đầu tiên, gấp sách và tự viết chương trình. Sau đó đọc "các cách chưa đúng": nếu chương trình của em trùng với một trong số đó, em đã biết mình mất bao nhiêu điểm và vì sao.

Cách tự đối chiếu với lời giải vét cạn

Ba tệp là đủ: chương trình của em (sol.cpp), chương trình vét cạn (trau.cpp hoặc trau.py) và một chương trình sinh test ngẫu nhiên nhỏ (gen.cpp, nhận một hạt giống từ dòng lệnh). Đoạn lệnh dưới đây chạy hàng trăm test và dừng lại ở test đầu tiên hai chương trình cho kết quả khác nhau.

‹› Đối chiếu tự động (chạy trên Linux hoặc Git Bash)

```
for i in $(seq 1 500); do
    ./gen $i > test.inp
    ./sol < test.inp > sol.out
    python3 trau.py < test.inp > trau.out
    if ! cmp -s sol.out trau.out; then echo "Khác nhau ở test $i"; break; fi
done
```

Với các bài số học, bộ sinh test nên trộn số ngẫu nhiên với **các giá trị đặc biệt**: 0, 1, 2, số nguyên tố, lũy thừa của 2, tích hai số nguyên tố lớn, p − 1, p, p + 1 (với p là modulo), giá trị lớn nhất trong giới hạn. Phần lớn lỗi về trường hợp đặc biệt lộ ra ngay ở những giá trị này.

Cách tự kiểm tra tốc độ và tràn số

Hãy tự tạo test lớn nhất có thể: n, m bằng giới hạn trên, số nguyên tố lớn nhất dưới giới hạn (chia thử sẽ chạy lâu nhất), và các số làm mọi phép nhân đạt giá trị lớn nhất. Biên dịch thêm với cờ -fsanitize=undefined để chương trình tự báo khi có phép tính tràn số.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CẤU TRÚC MỘT LỜI GIẢI **3**

CHƯƠNG 1. SÀNG NÂNG CAO **6**

Bài 1.1 NGUYÊN TỐ THỨ K 6
Bài 1.2 CẶP SONG SINH 9
Bài 1.3 TỔNG HAI NGUYÊN TỐ 11
Bài 2.1 SỐ ƯỚC CỦA TÍCH 13
Bài 2.2 BCNN DÃY SỐ 15
Bài 2.3 CẶP RỜI NHAU 17
Bài 3.1 NGUYÊN TỐ TRONG ĐOẠN 19
Bài 3.2 KHOẢNG CÁCH NGUYÊN TỐ 21
Bài 3.3 NHIỀU ƯỚC NHẤT TRONG ĐOẠN 23
Bài 4.1 TỔNG SỐ ƯỚC 25
Bài 4.2 SỐ DỒI DÀO 27
Bài 4.3 ĐẾM CẶP ƯCLN 29

CHƯƠNG 2. HÀM SỐ HỌC **31**

Bài 5.1 SỐ ƯỚC CỦA GIAI THỪA 31
Bài 5.2 TÍCH CÁC ƯỚC 34
Bài 5.3 SỐ NHỎ NHẤT CÓ K ƯỚC 36
Bài 6.1 PHI MỘT SỐ 38
Bài 6.2 CẶP NGUYÊN TỐ CÙNG NHAU 40
Bài 6.3 TỔNG UCLN 42
Bài 7.1 SÀNG MÖBIUS 44
Bài 7.2 KHÔNG CHÍNH PHƯƠNG 46
Bài 7.3 TỔNG HÀM NHÂN TÍNH 48

CHƯƠNG 3. MODULO NÂNG CAO **50**

Bài 8.1 LŨY THỪA MODULO 50
Bài 8.2 CHIA THEO MODULO 53
Bài 8.3 TỔNG CẤP SỐ NHÂN 55
Bài 9.1 BÉZOUT 57
Bài 9.2 PHÂN SỐ MODULO 59
Bài 9.3 ĐỒNG DƯ TRONG ĐOẠN 61
Bài 10.1 ĐỔI TIỀN 63
Bài 10.2 HAI CON ẾCH 65
Bài 10.3 HỆ ĐỒNG DƯ 67
Bài 11.1 NHÂN LỚN 69

Lê Thành Nam

Bài 11.2 NGUYÊN TỐ KẾ TIẾP 71
Bài 11.3 NGUYÊN TỐ TRONG ĐOẠN LỚN 73
Bài 12.1 LŨY THỪA KÉP 76
Bài 12.2 TÍCH CÁC ƯỚC LỚN 78
Bài 12.3 THÁP CAO 80

CHƯƠNG 4. TỔ HỢP MODULO P 83

Bài 13.1 ĐƯỜNG ĐI TRÊN LƯỚI 83
Bài 13.2 TỔNG NGHỊCH ĐẢO 86
Bài 13.3 LUCAS 88
Bài 14.1 CHIA QUÀ 90
Bài 14.2 ĐƯỜNG ĐI DƯỚI ĐƯỜNG CHÉO 92
Bài 14.3 DÃY NGOẶC 94
Bài 15.1 CÙNG NHAU VỚI M 96
Bài 15.2 CHIA KẸO CÓ GIỚI HẠN 98
Bài 15.3 SỐ THỨ K 100

CHƯƠNG 5. GIAI THỪA VÀ SỐ MŨ 102

Bài 16.1 CHỮ SỐ 0 TRONG HỆ b 102
Bài 16.2 TỔ HỢP CHIA HẾT 105
Bài 16.3 GIAI THỪA NHỎ NHẤT 107
Bài 17.1 SỐ 0 TẬN CÙNG NGƯỢC 109
Bài 17.2 CHỮ SỐ KHÁC 0 CỦA TÍCH 111
Bài 17.3 GIAI THỪA BỎ 113

CHƯƠNG 6. HAI KĨ THUẬT KHÁC 115

Bài 18.1 TỔNG FIBONACCI 115
Bài 18.2 TRUY HỒI BẬC K 118
Bài 18.3 ĐƯỜNG ĐI ĐỘ DÀI K 120
Bài 19.1 SỐ ĐẸP 122
Bài 19.2 ĐẾM CHỮ SỐ 124
Bài 19.3 SỐ TỐT THỨ K 126
Bài 20.1 CHIA HẾT CHO K 128
Bài 20.2 CHIA HẾT CHO MỌI CHỮ SỐ 130
Bài 20.3 TÍCH CHỮ SỐ 132

TỔNG KẾT: TỔNG KẾT: 60 BÀI, MỘT BẢNG ÔN TẬP 134

Lê Thành Nam

CHƯƠNG 1

SÀNG NÂNG CAO

Các lời giải của chương này cho thấy cùng một ý tưởng sàng có thể tốn 30 MB hay 3 MB, chạy 0,1 giây hay 10 giây, tùy cách cài đặt. Bảng điểm chỉ ra những lỗi rất hay gặp: sàng thiếu cận, gạch nhầm chính số nguyên tố, quên ước nguyên tố cuối cùng lớn hơn √n, và đếm cặp mà quên trừ phần chung.

Lê Thành Nam

Bài	Tên bài	Mã bài	Mức độ	Liên hệ lý thuyết
1.1	NGUYÊN TỐ THỨ K	NTTHUK	★ CƠ BẢN	Bài 1
1.2	CẶP SONG SINH	SONGSINH	★★ TRUNG BÌNH	Bài 1
1.3	TỔNG HAI NGUYÊN TỐ	GOLDBACH	★★★ NÂNG CAO	Bài 1
2.1	SỐ ƯỚC CỦA TÍCH	DEMUOC	★ CƠ BẢN	Bài 2
2.2	BCNN DÃY SỐ	BCNNDAY	★★ TRUNG BÌNH	Bài 2
2.3	CẶP RỜI NHAU	CAPROI	★★★ NÂNG CAO	Bài 2
3.1	NGUYÊN TỐ TRONG ĐOẠN	NTDOAN	★ CƠ BẢN	Bài 3
3.2	KHOẢNG CÁCH NGUYÊN TỐ	KCNT	★★ TRUNG BÌNH	Bài 3
3.3	NHIỀU ƯỚC NHẤT TRONG ĐOẠN	NHIEUUOC	★★★ NÂNG CAO	Bài 3
4.1	TỔNG SỐ ƯỚC	TONGUOC	★ CƠ BẢN	Bài 4
4.2	SỐ DỒI DÀO	DOIDAO	★★ TRUNG BÌNH	Bài 4
4.3	ĐẾM CẶP ƯCLN	CAPUCLN	★★★ NÂNG CAO	Bài 4

BÀI 1.1 NGUYÊN TỐ THỨ K

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
NTTHUK.INP	NTTHUK.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Bài 1 (sàng tuyến tính, cận trên của p_k)

1. Sàng tới đâu?

Câu hỏi đầu tiên của bài là **cận trên**: p_{500000} lớn bao nhiêu? Theo định lí về số nguyên tố, $p_k \approx k\cdot(\ln k + \ln\ln k - 1)$ và với $k \geq 6$ luôn có $p_k < k\cdot(\ln k + \ln\ln k)$. Với $k = 5\cdot10^5$: $\ln k \approx 13{,}12$, $\ln\ln k \approx 2{,}57$, nên $p_k < 5\cdot10^5\cdot15{,}7 \approx 7{,}85\cdot10^6$. Thực tế $p_{500000} = 7368787$. Ta sàng một lần tới $N = 7{,}4\cdot10^6$, lưu các số nguyên tố vào mảng pr theo thứ tự tăng dần, và mỗi câu hỏi chỉ là in pr[k − 1].

2. Các cách chưa đúng

■ ĐIỂM BẾ TẮC

- **chiathu.cpp** đếm dần từng số nguyên tố bằng chia thử tới $\sqrt{x}$ cho mỗi câu hỏi. Với $k \leq 1000$ thì $x \leq 7919$, rất nhanh; với $k = 5\cdot10^4$ mỗi câu hỏi đã cần hàng chục triệu phép chia.
- **sanglai.cpp** sàng Eratosthenes tới cận trên của k **cho mỗi câu hỏi**. Mỗi lần sàng chỉ mất vài phần trăm giây, đủ cho $q \leq 100$, nhưng 10^5 câu hỏi thì phải sàng 10^5 lần.
- **giuhan.cpp** sàng một lần nhưng "cho chắc" chỉ tới 10^6: chỉ có 78498 số nguyên tố nhỏ hơn 10^6, nên mọi câu hỏi $k > 78498$ đều sai. Lỗi này rất hay gặp khi không ước lượng cận trên trước.

3. Cách tối ưu

NTTHUK.CPP: Sàng tuyến tính một lần tới 7,4·10^{6}

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 7400000;                    // p_500000 = 7368787 < N
int lp[N + 1];
vector<int> pr;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("NTTHUK.INP", "r")) {
        freopen("NTTHUK.INP", "r", stdin);
        freopen("NTTHUK.OUT", "w", stdout);
    }
    for (int i = 2; i <= N; i++) {        // sàng tuyến tính
        if (lp[i] == 0) {
            lp[i] = i;
            pr.push_back(i);
        }
        for (int p : pr) {
            if (p > lp[i] || (long long)i * p > N) break;
            lp[i * p] = p;
        }
    }
    int q;
    cin >> q;
    while (q--) {
        int k;
```

Lê Thành Nam

NTTHUK.CPP: Sàng tuyến tính một lần tới 7,4·10^{6}

```
29          cin >> k;
30          cout << pr[k - 1] << '\n';
31      }
32      return 0;
33  }
```

Độ phức tạp: O(N) cho sàng và O(1) cho mỗi câu hỏi. Bộ nhớ: mảng lp gồm $7{,}4 \cdot 10^6$ số int (khoảng 30 MB) và mảng pr gồm $5 \cdot 10^5$ số.

4. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Chia thử từng số cho mỗi câu hỏi	30	0	0	30
Sàng lại từ đầu cho mỗi câu hỏi	30	30	0	60
Sàng một lần nhưng chỉ tới 10^6	30	30	0	60
Sàng tuyến tính một lần tới $7{,}4 \cdot 10^6$	**30**	**30**	**40**	**100**

Lê Thành Nam

BÀI 1.2 CẶP SONG SINH

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
SONGSINH.INP	SONGSINH.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Bài 1 (sàng, mảng cộng dồn)

1. Mảng cộng dồn

Sàng một lần tới 10^7. Gọi c[x] là số các $p \le x$ sao cho p và p + 2 đều nguyên tố. Khi đó c[x] = c[x − 1] + (1 nếu x và x + 2 nguyên tố). Cặp (p, p + 2) nằm trọn trong [a, b] khi và chỉ khi $a \le p \le b - 2$, nên đáp số là c[b − 2] − c[a − 1] (bằng 0 nếu b − 2 < a).

■ BẪY THƯỜNG GẶP

Điều kiện "nằm trọn" là $p \le b - 2$, không phải $p \le b$. Chương trình **biến.cpp** lấy c[b] − c[a − 1] nên đếm thừa cặp có p = b − 1 hoặc p = b (ví dụ [6, 11] với cặp (11, 13)). Với một vài câu hỏi ngẫu nhiên, b hiếm khi rơi đúng vào p hoặc p + 1 của một cặp nên chương trình qua được hai subtask đầu; nhưng với 10^5 câu hỏi thì gần như chắc chắn có câu rơi vào trường hợp đó, và cả subtask 3 bị mất điểm.

2. Các cách chậm

■ ĐIỂM BẾ TẮC

- **chiathu.cpp** kiểm tra từng p trong đoạn bằng chia thử: $O((b - a)\cdot\sqrt{b})$ cho mỗi câu hỏi, chỉ chạy kịp khi $b \le 10^4$.
- **duyet.cpp** sàng một lần nhưng mỗi câu hỏi vẫn duyệt cả đoạn [a, b]: $O(q\cdot 10^7)$, được khi $q \le 10$.

3. Cách tối ưu

SONGSINH.CPP: Sàng một lần + mảng cộng dồn

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 10000002;
char hop[N + 1];                          // hop[x] = 1 nếu x không nguyên tố
// c[x] = số p ≤ x có p, p + 2 nguyên tố
int c[N + 1];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("SONGSINH.INP", "r")) {
```

Lê Thành Nam

SONGSINH.CPP: Sàng một lần + mảng cộng dồn

```
        freopen("SONGSINH.INP", "r", stdin);
        freopen("SONGSINH.OUT", "w", stdout);
    }
    hop[0] = hop[1] = 1;
    for (long long i = 2; i * i <= N; i++)
        if (!hop[i])
            for (long long j = i * i; j <= N; j += i) hop[j] = 1;
    for (int x = 1; x + 2 <= N; x++)
        c[x] = c[x - 1] + (!hop[x] && !hop[x + 2]);
    int q;
    cin >> q;
    while (q--) {
        int a, b;
        cin >> a >> b;
        // p chạy trong [a, b - 2]
        cout << (b - 2 >= a ? c[b - 2] - c[a - 1] : 0) << '\n';
    }
    return 0;
}
```

Độ phức tạp: O(N) chuẩn bị, O(1) mỗi câu hỏi. Mảng sàng dùng kiểu char (10 MB) và mảng cộng dồn kiểu int (40 MB).

4. Bảng điểm theo thuật toán

Thuật toán	ST1 (20)	ST2 (30)	ST3 (50)	Tổng
Chia thử từng số trong đoạn	20	0	0	20
Sàng một lần, duyệt cả đoạn cho mỗi câu hỏi	20	30	0	50
Cộng dồn nhưng quên điều kiện p + 2 ≤ b	20	30	0	50
Sàng một lần + mảng cộng dồn	**20**	**30**	**50**	**100**

Lê Thành Nam